

第3部分 竞赛篇

第8章 高效算法设计

学习目标

- ☑ 理解“基本操作”、渐进时间复杂度的概念和大O记号的含义
- ☑ 掌握“最大连续和”问题的各种算法及其时间复杂度分析
- ☑ 正确认识算法分析的优点和局限性，能正确使用分析结果
- ☑ 掌握归并排序和逆序对统计的分治算法
- ☑ 理解快速排序和快速选择算法
- ☑ 熟练掌握二分查找算法，包括找上下界的算法
- ☑ 能用递归的方式思考和求解问题
- ☑ 熟练掌握用二分法求解非线性方程的方法
- ☑ 熟练掌握用二分法把优化问题转化为判定问题的方法
- ☑ 熟悉能用贪心法求解的各类经典问题
- ☑ 掌握本章中介绍的各种算法设计思路与方法

尽管直观、适用范围广，但枚举、回溯等暴力方法常常无法走出“低效”的阴影。这并不难理解：越是通用的算法，越不能深入挖掘问题的特殊性。本章介绍一些经典问题的高效算法。由于是“量身定制”的，这些算法从概念、思路到程序实现都是千差万别的。从某种意义上说，从本章开始，读者才刚刚开始接触“严肃”的算法设计理论。

8.1 算法分析初步

编程者都希望自己的算法高效，但算法在写成程序之前是运行不了的。难道每设计出来一个算法都必须写出程序来才知道快不快吗？答案是否定的。本节介绍算法分析的基本概念和方法，力求在编程之前尽量准确地估计程序的时空开销，并作出决策——例如，如果算法又复杂速度又慢，就不要急着写出来了。

8.1.1 渐进时间复杂度

最大连续和问题。给出一个长度为 n 的序列 $A_1, A_2, \dots, A_n$ ，求最大连续和。换句话说，要求找到 $1 \leq i \leq j \leq n$ ，使得 $A_i + A_{i+1} + \dots + A_j$ 尽量大。

【分析】

使用枚举，得出如下程序：

程序 8-1 最大连续和 (1)

```
tot = 0;
best = A[1]; //初始最大值
for(int i = 1; i <= n; i++)
    for(int j = i; j <= n; j++){ //检查连续子序列 A[i], ..., A[j]
        int sum = 0;
        for(int k = i; k <= j; k++) { sum += A[k]; tot++; } //累加元素和
        if(sum > best) best = sum; //更新最大值
    }
```

注意 best 的初值是 A[1]，这是最保险的做法——不要写 best=0（想一想，为什么）。当 $n=1000$ 时，输出 tot=167167000，这是加法运算的次数。当 $n=50$ 时，输出 22100。

为什么要计算 tot 呢？因为它与机器的运行速度无关。不同机器的速度不一样，运行时间也会有所差异，但 tot 值一定相同。换句话说，它去掉了机器相关的因素，只衡量算法的“工作量”大小——具体来说，是“加法”操作的次数。

提示 8-1：统计程序中“基本操作”的数量，可以排除机器速度的影响，衡量算法本身的优劣程度。

在本题中，将“加法操作”作为基本操作，类似地也可以把其他四则运算、比较运算作为基本操作。一般并不会严格定义基本操作的类型，而是根据不同情况灵活处理。

刚才实验得出 tot 值的，其实它也可以用数学方法直接推导出。设输入规模为 n 时加法操作的次数为 $T(n)$ ，则：

$$T(n) = \sum_{i=1}^n \sum_{j=i}^n j - i + 1 = \sum_{i=1}^n \frac{(n-i+1)(n-i+2)}{2} = \frac{n(n+1)(n+2)}{6}$$

上面的公式是关于 n 的三次多项式，意味着当 n 很大时，平方项和一次项对整个多项式值的影响不大。可以用一个记号来表示： $T(n) = \Theta(n^3)$ ，或者说 $T(n)$ 和 n^3 同阶。

同阶是什么意思呢？简单地说，就是“增长情况相同”。前面说过， n 很大时，只有立方项起到决定作用，而立方项的系数对“增长”是不起作用的—— n 扩大两倍时， n^3 和 $100n^3$ 都扩大 8 倍。这样一来，可以只保留“最大项”，并忽略其系数，得到的简单式子称为算法的渐进时间复杂度（asymptotic time complexity）。

提示 8-2：基本操作的数量往往可以写成关于“输入规模”的表达式，保留最大项并忽略系数后的简单表达式称为算法的渐进时间复杂度，用于衡量算法中基本操作数随规模的增长情况。

读者可以做个实验，看看 n 扩大两倍时运行时间是否近似扩大 8 倍。注意这里的“8 倍”是近似的，因为在 $T(n)$ 的表达式中，二次项、一次项和常数项都被忽略掉了；程序中的其他运算，如 if(sum > best) 中的比较运算，甚至改变循环变量所需的“自增”都没有考虑在内。

尽管如此，算法分析的效果还是比较精确的，因为抓住了主要矛盾——执行得最多的运算是加法。

提示 8-3：渐进时间复杂度忽略了很多因素，因而分析结果只能作为参考，并不是精确的。尽管如此，如果成功抓住了最主要的运算量所在，算法分析的结果常常十分有用。

8.1.2 上界分析

对于上面的方法，读者可能会有疑问：难道每次都要作一番复杂的数学推导才能得到渐进时间复杂度吗？当然不必。

下面是另外一种推导方法：算法包含 3 重循环，内层最坏情况下需要循环 n 次，中层循环最坏情况下也需要 n 次，外层循环最坏情况下仍然需要 n 次，因此总运算次数不超过 n^3 。这里采用了“上界分析”，假定所有最坏情况同时取到，尽管这是不可能的。不难预料，这样的分析和实际情况肯定会有一定偏差——在 $T(n)$ 的表达式中， n^3 的系数是 $1/6$ ，小于 n^3 ，但数量级是正确的——仍然可以得到“ n 扩大两倍时，运行时间近似扩大 8 倍”的结论。上界也有记号： $T(n)=O(n^3)$ 。

提示 8-4：在算法设计中，常常不进行精确分析，而是假定各种最坏情况同时取到，得到上界。在很多情况下，这个上界和实际情况同阶（称为“紧”的上界），但也有可能会因为分析方法不够好，得到“松”的上界。

松的上界也是正确的上界，但可能让人过高估计程序运行的实际时间（从而不敢编写程序），而即使上界是紧的，过大（如 100）或过小（如 $1/100$ ）的最高项系数同样可能引起错误的估计。换句话说，算法分析不是万能，要谨慎对待分析结果。如果预感到上界不紧、系数过大或者过小，最好还是要编程实践。

下面试着优化一下这个算法。设 $S_i=A_1+A_2+\cdots+A_i$ ，则 $A_i+A_{i+1}+\cdots+A_j=S_j-S_{i-1}$ 。该式子的用途相当广泛，其直观含义是“连续子序列之和等于两个前缀和之差”。有了这个结论，最内层的循环就可以省略了。

程序 8-2 最大连续和（2）

```
S[0] = 0;
for(int i = 1; i <= n; i++) S[i] = S[i-1] + A[i];           //递推前缀和 S
for(int i = 1; i <= n; i++)
    for(int j = i; j <= n; j++) best = max(best, S[j]-S[i-1]); //更新最大值
```

注意上面的程序用到了递推的思想：从小到大依次计算 $S[1], S[2], S[3], \dots$ ，每个只需要在前一个的基础上加上一个元素。换句话说，“计算 S ”这个步骤的时间复杂度为 $O(n)$ 。接下来是一个二重循环，用类似的方法可以分析出：

$$T(n) = \sum_{i=1}^n n-i+1 = \frac{n(n+1)}{2}$$

代入可得 $T(1000)=500500$ ，和运行结果一致。同样地，用上界分析可以更快地得到结

论：内层循环最坏情况下要执行 n 次，外层也是，因此时间复杂度为 $O(n^2)$ 。

8.1.3 分治法

本节使用分治法来解决这个问题。分治算法一般分为如下 3 个步骤。

划分问题：把问题的实例划分成子问题。

递归求解：递归解决子问题。

合并问题：合并子问题的解得到原问题的解。

在本例中，“划分”就是把序列分成元素个数尽量相等的两半；“递归求解”就是分别求出完全位于左半或者完全位于右半的最佳序列；“合并”就是求出起点位于左半、终点位于右半的最大连续和序列，并和子问题的最优解比较。

前两部分没有什么特别之处，关键在于“合并”步骤。既然起点位于左半，终点位于右半，则可以人为地把这样的序列分成两部分，然后独立求解：先寻找最佳起点，然后再寻找最佳终点。

程序 8-3 最大连续和 (3) (如图 8-1 所示)

```
int maxsum(int* A, int x, int y){ //返回数组在左闭右开区间[x,y)中的最大连续和
    int v, L, R, maxs;
    if(y - x == 1) return A[x]; //只有一个元素，直接返回
    int m = x + (y-x)/2; //分治第一步：划分成[x, m)和[m, y)
    int maxs = max(maxsum(A, x, m), maxsum(A, m, y)); //分治第二步：递归求解
    int v, L, R;
    v = 0; L = A[m-1]; //分治第三步：合并(1)——从分界点开始往左的最大连续和L
    for(int i = m-1; i >= x; i--) L = max(L, v += A[i]);
    v = 0; R = A[m]; //分治第三步：合并(2)——从分界点开始往右的最大连续和R
    for(int i = m; i < y; i++) R = max(R, v += A[i]);
    return max(maxs, L+R); //把子问题的解与L和R比较
}
```

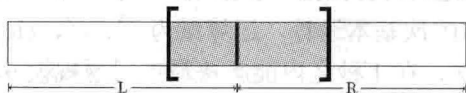


图 8-1 最大连续和的分治算法

上面的代码用到了“赋值运算本身具有返回值”的特点，在一定程度上简化了代码，但不会牺牲可读性。

在上面的程序中， L 和 R 分别为从分界线往左、往右能达到的最大连续和。对于 $n=1000$ ， tot 值仅为 9976，在前面的 $O(n^2)$ 算法基础上又有大幅度改进。

是否可以像前面那样，得到 tot 的数学表达式呢？注意求和技巧已经不再适用，需要用递归的思路进行分析：设序列长度为 n 时的 tot 值为 $T(n)$ ，则 $T(n)=2T(n/2)+n$ ， $T(1)=1$ 。其中 $2T(n/2)$ 是两次长度为 $n/2$ 的递归调用，而最后的 n 是合并的时间（整个序列恰好扫描一遍）。注意这个方程是近似的，因为当 n 为奇数时两次递归的序列长度分别为 $(n-1)/2$ 和

$(n+1)/2$ ，而不是 $n/2$ 。幸运的是，这样的近似对于最终结果影响很小，在分析算法时总是可以忽略它。

提示 8-5：在算法分析中，往往可以忽略“除法结果是否为整数”，而直接按照实数除法分析。这样的近似对最终结果影响很小，一般不会改变渐进时间复杂度。

解刚才的方程，可以得到 $T(n) = \Theta(n \log n)$ 。由于 $n \log n$ 增长很慢，当 n 扩大两倍时，运行时间的扩大倍数只是略大于 2。现在不必懂得解方程的方法，可以把它作为一个重要结论记下来（建议有兴趣的读者试着借助于解答树来证明这个结论，它并不复杂）。

提示 8-6：递归方程 $T(n) = 2T(n/2) + \Theta(n)$ ， $T(1)=1$ 的解为 $T(n) = \Theta(n \log n)$ 。

在结束对分治算法的讨论之前，有必要再谈谈上述程序中的两个细节。首先是范围表示。上面的程序用左闭右开区间来表示一个范围，好处是在处理“数组分割”时比较自然：区间 $[x, y)$ 被分成的是 $[x, m)$ 和 $[m, y)$ ，不需要在任何地方加减 1。另外，空区间表示为 $[x, x)$ ，比 $[x, x-1]$ 顺眼多了。

另一个细节是“分成元素个数尽量相等的两半”时分界点的计算。在数学上，分界点应当是 x 和 y 的平均数 $m=(x+y)/2$ ，此处用的却是 $x+(y-x)/2$ 。在数学上二者相等，但在计算机中却有差别。不知读者是否注意到，运算符“/”的“取整”是朝零方向（towards zero）的取整，而不是向下取整。换句话说， $5/2$ 的值是 2，而 $-5/2$ 的值是 -2。为了方便分析，此处用 $x+(y-x)/2$ 来确保分界点总是靠近区间起点。这在本题中并不是必要的，但在后面要介绍的二分查找中，却是相当重要的技巧。

8.1.4 正确对待算法分析结果

对于“最大连续和”问题，本书先后介绍了时间复杂度为 $O(n^3)$ 、 $O(n^2)$ 、 $O(n \log n)$ 的算法，每个新算法较前一个来说，都是重大的改进。尽管分治法看上去很巧妙，但并不是最高效的。把 $O(n^2)$ 算法稍作修改，便可以得到一个 $O(n)$ 算法：当 j 确定时，“ $S[j]-S[i-1]$ 最大”相当于“ $S[i-1]$ 最小”，因此只需要扫描一次数组，维护“目前遇到过的最小 S ”即可。

假设机器速度是每秒 10^8 次基本运算，运算量为 n^3 、 n^2 、 $n \log_2 n$ 、 n 、 2^n （如子集枚举）和 $n!$ （如排列枚举）的算法，在 1 秒之内能解决最大问题规模 n ，如表 8-1 所示。

表 8-1 运算量随着规模的变化

运算量	$n!$	2^n	n^3	n^2	$n \log_2 n$	n
最大规模	11	26	464	10000	4.5×10^6	100000000
速度扩大两倍后	11	27	584	14142	8.6×10^6	200000000

表 8-1 还给出了机器速度扩大两倍后，算法所能解决规模的对比。可以看出， $n!$ 和 2^n 不仅能解决的问题规模非常小，而且增长缓慢；最快的 $n \log_2 n$ 和 n 算法不仅解决问题的规模大，而且增长快。渐进时间复杂为多项式的算法称为多项式时间算法（polynomial-time algorithm），也称有效算法；而 $n!$ 或者 2^n 这样的低效的算法称为指数时间算法（exponential-time algorithm）。